Portfolio Conflict Resolution in Concurrent Multi-Agent Large Neighborhood Search for Coupled Multiple-Choice Allocation

Naresh R[a,*]

[a]*Birla Institute of Technology and Science, Pilani (BITS Pilani), Pilani, Rajasthan, India*

ARTICLE INFO

Keywords:
Large neighborhood search
Parallel metaheuristics
Multiple-choice knapsack
Conflict resolution
Matheuristics
Block-angular integer programming

ABSTRACT

When several large neighborhood search (LNS) workers improve a block-angular allocation problem at the same time, their moves compete for shared capacities and must be reconciled at a synchronization barrier. We study this barrier for a multiple-choice allocation problem with shared hub capacities. In the proposed portfolio-coordinated multi-agent LNS, each worker sends four candidate moves that use different amounts of shared capacity, and the barrier selects one candidate per worker, exactly by a small integer program or greedily. In a controlled study with shared operators, five runs per instance and exact tests, portfolio coordination beats optimistic rollback and sequential-lock commit, even when these rules receive the same portfolios (56 to 58 of 60 instances), and beats static and demand-proportional quotas on all 60. The exact barrier gains about 20% more per conflict epoch than greedy resolution by gain per unit of capacity, yet this greedy rule finishes slightly but significantly better in all six test families (0.03 to 0.44 percentage points). With four processes and budgets of 0.5 to 6 s, the method beats a mixed-integer solver given the same wall-clock time on all 80 instances with 20 or 40 candidates per request and on the larger six-candidate instances; with five seconds or equal CPU time the solver is better on most six-candidate instances. Shadow prices and dynamic quotas give no measurable gain. On 27 public multiple-choice knapsack benchmarks it stalls far from the best known solutions, a limit of improvement-only workers.

1. Introduction

Planning problems in multi-depot logistics often have a block-angular structure: K regional subproblems (depots, zones or agents) with their own local constraints, linked by a few shared capacities such as transfer hubs. Large neighborhood search (LNS) (Shaw, 1998) and adaptive LNS (Røpke & Pisinger, 2006) are standard metaheuristics for such problems, and several authors run LNS with several workers at the same time to obtain good solutions under short time limits (Røpke, 2009; Bartodziej, Derigs, & Vogel, 2010; Asokan, Furman, Goel, Li, & Shao, 2014; Chan, Chen, Lin, Zhang, Harabor, Koenig, Huang, & Phan, 2024). When the workers share capacities, their moves must be reconciled at a synchronization point without violating those capacities.

Two families of commit rules are used for this. Under *optimistic commit*, every worker repairs its subproblem as if the others stayed at the incumbent, so it may use the whole residual slack $s^{(t)}$ of every shared capacity; if the merged moves do not fit, a batch rule rolls them back (`Opt-Rollback`), or a sequential lock accepts them in arrival order while they fit (`Opt-SeqLock`), as in operation-parallel LNS (Chan et al., 2024). Under *static quotas*, every worker receives the share $s^{(t)}/K$; every merge is feasible, but a move that needs more than its share is never proposed, even if other workers leave theirs unused. Optimistic commit loses proposals, quotas leave capacity idle, and neither lets the coordinator trade capacity between workers.

This paper studies a third option. In *portfolio-coordinated multi-agent LNS* (`PC-MALNS`), each worker sends a small portfolio of candidate moves that differ in how much shared capacity they consume: a capacity-releasing or unchanged incumbent, a quota-safe repair, a repair that may borrow capacity at a penalty, and the optimistic repair of the baselines. The barrier then chooses one candidate per worker so

*Corresponding author. ORCID: orcid.org/0009-0000-0735-2066. Source code and benchmark data: github.com/Naresh23032003/DG-MALNS.

rnaresh23032003r@gmail.com (N. R)

ORCID(s): 0009-0000-0735-2066 (N. R)

that the selection fits into $s^{(t)}$. We compare two families of barrier rules on the same portfolios: an exact multiple-choice conflict-resolution integer program (`MC-CRIP`), which maximizes the cost reduction of the current epoch, and polynomial greedy rules that sort upgrades by gain (`Greedy-Gain-Portfolio`) or by gain per unit of capacity (`Greedy-Efficiency-Portfolio`, `BCRM-Greedy`).

The contributions are as follows.

1. *Method and theory.* We formalize the portfolio barrier as a multiple-choice multidimensional knapsack problem over the workers' candidates and show that the worker quotas cancel in the barrier constraint, so they act only as worker-side trust regions that diversify the portfolio. Every epoch is feasible with monotone descent, the exact barrier dominates every other rule on a common portfolio in each epoch, and we give worst-case single-epoch ratios of the other rules (Propositions 1 and 2).
2. *Controlled evaluation with real parallel execution.* All LNS methods share the same worker code, operators, random streams and warm start; every stochastic method has five independent runs per instance; and all parallel runs use a persistent pool of worker processes with real wall-clock budgets. The configuration was fixed on validation instances that are disjoint from the test instances.
3. *Findings.* (i) Portfolio coordination beats optimistic rollback and sequential-lock commit, also when these rules receive the same portfolio, and beats static and demand-proportional quotas. (ii) Counterfactual logging on 1,955 conflict epochs confirms that the exact barrier gains more per epoch, yet the greedy efficiency rule finishes slightly but significantly better over whole runs; we report this result although it does not favour the exact barrier, and we explain why the exact barrier remains the pre-registered primary mode. (iii) Under real-time budgets of 0.5 to 6 s on four processes, `PC-MALNS` beats wall-clock-matched HiGHS when each request has 20 or 40 candidates, and on the larger instances with six candidates; with longer limits or equal CPU time HiGHS is better on six-candidate instances. (iv) Shadow prices and dynamic quotas add nothing measurable. (v) On public multiple-choice multidimensional knapsack (MMKP) instances the method stalls, which identifies a structural limit of improvement-only workers.

Section 2 positions the work, Section 3 states the model, the method and its properties, Section 4 describes the experimental design, Section 5 reports the results, and Section 6 concludes.

2. Literature review and positioning

LNS and adaptive LNS. Shaw (1998) introduced LNS for vehicle routing; Røpke & Pisinger (2006) developed adaptive LNS for the pickup and delivery problem with time windows, with regret insertion going back to Potvin & Rousseau (1993), and Pisinger & Røpke (2007) generalized it to a broad family of routing problems. Adaptive LNS has been applied to pickup and delivery with transfers (Masson, Lehuédé, & Péton, 2013) and two-echelon city logistics (Hemmelmayr, Cordeau, & Crainic, 2012), and Hendel (2022) extended it to general mixed-integer programs. Santini, Røpke, & Hvattum (2018) compare acceptance criteria; all methods in this paper use greedy acceptance. Construct, Merge, Solve & Adapt (Blum, Pinacho, López-Ibáñez, & Lozano, 2016) solves a restricted integer program over solution components collected from constructed solutions; our barrier program is a restricted master over the candidate moves of concurrent workers.

Parallel and distributed LNS. Parallel metaheuristics and cooperative search are surveyed by Alba (2005), Crainic (2019) and Schryen (2020); examples in routing include Groër, Golden, & Wasil (2011) and Lahrichi, Crainic, Gendreau, Rei, Crisan, & Vidal (2015). Røpke (2009) describes `PALNS`, a general software framework in which worker threads apply destroy-and-repair moves to a shared incumbent protected by locks; it does not use batch rollback, and our `Opt-Rollback` rule is a batch adaptation, not the `PALNS` implementation. Bartodziej et al. (2010) report first results on parallel LNS for rich pickup-and-delivery problems, and Asokan et al. (2014) study deterministic and non-deterministic parallel LNS for liquefied natural gas inventory routing. In multi-agent path finding, Li, Chen, Harabor, Stuckey, & Koenig (2021) propose anytime LNS and Chan et al. (2024) an operation-parallel variant (`DROP-LNS`) whose workers' proposals are validated sequentially under locks; our `Opt-SeqLock` rule adapts this sequential validation to shared capacities. In distributed constraint optimization (Fioretto, Pontelli, & Yeoh, 2018), Fioretto, Dovier, Pontelli, Yeoh, & Zivan (2017) and Hoang, Fioretto, Yeoh, Pontelli, & Zivan (2018) let agents destroy and repair local

Table 1
Positioning relative to related work. `Opt-Rollback`, `Opt-SeqLock` and `Static-Quota` are our adaptations of the respective synchronization ideas to shared capacities, not the original implementations.

Study	Problem	Parallelism	Shared capacities	Conflict resolution	Feasible merge
Røpke & Pisinger (2006)	PDPTW	Sequential	Within one model	Not needed	Yes
Røpke (2009) (`PALNS`)	General LNS framework	Threads, shared incumbent	Not modelled	Locks on the incumbent	Not addressed
Bartodziej et al. (2010)	Rich pickup and delivery	Parallel LNS	Not modelled	Not described	Not addressed
Asokan et al. (2014)	LNG inventory routing	Parallel LNS	Not modelled	Not addressed	Not addressed
Fioretto et al. (2017)	DCOP	Multi-agent, synchronous	No hard shared capacities	Distributed repair	Local constraints
Hoang et al. (2018)	DCOP	Multi-agent, synchronous	No hard shared capacities	Distributed repair	Local constraints
Chan et al. (2024) (`DROP-LNS`)	Multi-agent path finding	Operation parallel	Collision constraints	Sequential validation	Yes (rejects)
Blum et al. (2016) (CMSA)	General CO	Sequential	Within one model	Restricted ILP	Yes
Falsone et al. (2017)	Distributed MILP	Multi-agent	Linear coupling	Dual decomposition	Asymptotically
This paper (`PC-MALNS`)	**Multiple-choice allocation**	**Processes, synchronous**	**Hub capacities**	**Portfolio barrier**	**Yes (Prop. 1)**

neighborhoods concurrently; their models have no shared resource capacities of the kind studied here, and the static quota rule we compare with is our adaptation, not a rule of these papers.

Decomposition and prices. Lagrangian relaxation (Held, Wolfe, & Crowder, 1974; Fisher, 1981; Guignard, 2003), subgradient methods (Polyak, 1969) and augmented Lagrangian and splitting schemes (Bertsekas, 1982; Boyd, Parikh, Chu, Peleato, & Eckstein, 2011) price the coupling constraints of block-angular programs, but rarely yield feasible integer solutions when the subproblems are solved independently. Vujanic, Mohajerin Esfahani, Goulart, Mariéthoz, & Morari (2016) recover feasible solutions of large block-angular mixed-integer programs by tightening the coupling constraints, and Falsone, Margellos, Garatti, & Prandini (2017) give a distributed dual-decomposition scheme with coupling constraints for multi-agent problems. Resource-directive decomposition (Geoffrion, 1970) allocates the shared resources to the subsystems, as quotas do. Restricted-master heuristics select among generated columns (Joncour, Michel, Sadykov, Sverdlov, & Vanderbeck, 2010; Sadykov, Vanderbeck, Pessoa, Tahiri, & Uchoa, 2019).

Multiple-choice multidimensional knapsack. The barrier program is an MMKP. Heuristics include Khan, Li, Manning, & Akbar (2002), Hifi, Michrafy, & Sbihi (2004) and the reactive local search of Hifi, Michrafy, & Sbihi (2006); the reduce-and-solve method of Chen & Hao (2014), the iterative pseudo-gap enumeration of Gao, Lu, Yao, & Li (2017), the core-based exact algorithm of Mansini & Zanotti (2020) and the two-phase kernel search of Lamanna, Mansini, & Zanotti (2022) are state of the art on the classical benchmarks that we use in Section 5.4. We use these instances only to locate the limits of concurrent LNS, not to compete with these dedicated methods. Anytime performance is measured by the primal integral of Berthold (2013).

Table 1 positions `PC-MALNS` relative to the closest studies.

3. Model and method

3.1. Route-pattern allocation with shared hubs (RPA-SH)

There are K agents (regional depots) $k \in \mathbf{K} = \{1, \dots, K\}$. Agent k serves the requests $i \in \mathbf{I}_k = \{1, \dots, N_{\text{agent}}\}$, $N = KN_{\text{agent}}$ in total, and assigns every request to exactly one of P pre-generated route-schedule patterns $p \in \mathbf{P} = \{0, \dots, P-1\}$: $p = 0$ is a direct dispatch that uses no shared capacity, and $p \geq 1$ are consolidated routes through shared transfer hubs. Pattern p of request (k, i) has cost $c_{k,i,p}$, uses $D_{r,k,i,p} \geq 0$ of hub capacity $r \in \{1, \dots, m\}$ and $A_{w,k,i,p} \geq 0$ of the local dispatch wave $w \in \{1, \dots, m_{\text{local}}\}$ of its depot. Precedence, time windows and vehicle sequencing are assumed to be satisfied by every pre-generated pattern and are not modelled; the problem is a restricted-master allocation over these patterns, i.e. a block-angular MMKP. With $x_{k,i,p} \in \{0, 1\}$ and the agent vectors x_k,

$$\min \quad C(x) = \sum_{k=1}^{K} c_k^{\top} x_k \tag{1}$$

$$\text{s.t.} \quad \sum_{k=1}^{K} D_k x_k \leq d, \tag{2}$$

$$x_k \in \mathbf{X}_k = \left\{ x_k \in \{0,1\}^{N_{\text{agent}} P} : A_k x_k \leq b_k,\ \textstyle\sum_p x_{k,i,p} = 1\ \forall i \in \mathbf{I}_k \right\}, \quad k \in \mathbf{K}, \tag{3}$$

Table 2
Notation.

Symbol	Meaning
K, k, j	number of agents (workers), agent indices
N_{agent}, i, P, p, p_i	requests per agent, request, number of patterns, pattern, current pattern of request i
m, r; m_{local}, w	number of hubs, hub index; number of local waves, wave index
c_k, D_k ($D_{k,i,p}$), A_k	costs, hub usage (column of pattern p of request i), wave usage of agent k
d, b_k	hub capacities, wave capacities of depot k
$x^{(t)}$, $s^{(t)} = d - \sum_k D_k x_k^{(t)}$	incumbent and residual hub slack at epoch t
$\Omega_k(x_k)$, x_k^{part}	destroy neighborhood of x_k and destroyed (partial) solution
M	chained destroy–repair steps per portfolio level
$\alpha_k^{(t)}$, $q_k^{(t)} = \alpha_k^{(t)} \circ s^{(t)}$	quota shares of agent k ($\sum_k \alpha_{r,k}^{(t)} = 1$) and quota
L, l, $y_k^{(t,l)}$	highest portfolio level ($L = 3$), level, level-l candidate of agent k
$\Delta c_{k,l}^{(t)}$, $\Delta D_{k,l}^{(t)}$, $\delta_{k,l}^{(t)}$	cost change, hub usage change and quota-relative usage change of level l
ρ_t	weight of the quadratic quota-excess penalty
$\lambda^{(t)}$	optional hub prices (ablation only; $\lambda^{(t)} = 0$ in `PC-MALNS`)
π	arrival permutation of the workers (`Opt-SeqLock`)
$e_{k,l}$, $\phi_{k,l}$	efficiency score of `Greedy-Efficiency-Portfolio` and ranking score of `BCRM-Greedy`
σ	standard deviation of the Regret-2 cost noise
$T_{\max}$	last epoch of a run (the budget is reached)

where $D_k \in \mathbb{R}_+^{m \times N_{\text{agent}} P}$ has the columns $D_{k,i,p} = (D_{r,k,i,p})_r$, $d \in \mathbb{R}_+^m$ are the hub capacities and b_k the wave capacities of depot k. We write p_i for the pattern of request i in the current solution of its agent. Table 2 lists the notation.

3.2. Worker portfolio generation

`PC-MALNS` starts from the `LP-Round` solution $x^{(0)}$: the LP relaxation of (1)–(3) is solved once, patterns with positive LP value are rounded greedily in decreasing order of reduced-cost saving subject to d and b_k, and one pass gives every request still on $p = 0$ its cheapest feasible pattern. Its time is charged to every run. In epoch $t = 0, 1, \ldots, T_{\max}$, the coordinator sends every worker the residual slack $s^{(t)}$ and its quota $q_k^{(t)} = s^{(t)}/K$ (uniform shares $\alpha_{r,k}^{(t)} = 1/K$). Worker k destroys 16% to 30% of its requests, $x_k^{\text{part}} \in \Omega_k(x_k^{(t)})$, and builds the portfolio $\mathbf{P}_k^{(t)} = \{y_k^{(t,0)}, \ldots, y_k^{(t,3)}\}$ (Figure 1):

- *Level* 0 *(incumbent or donor):* $y_k^{(t,0)} = x_k^{(t)}$, unless a donor rebalance of $x_k^{(t)}$ costs no more, uses no more capacity on any hub and less on at least one; then the rebalance is used. Level 0 is built from $x_k^{(t)}$, not from x_k^{part}.
- *Level* 1 *(quota-safe repair):* Regret-2 repair of x_k^{part} on the true costs subject to $D_k(y - x_k^{(t)}) \le q_k^{(t)}$.
- *Level* 2 *(borrower repair):* Regret-2 repair of x_k^{part} that minimizes the quota-excess surrogate

$$\tilde{F}_k(y) = c_k^\top y + \left(\lambda^{(t)}\right)^\top \left[D_k(y - x_k^{(t)}) - q_k^{(t)}\right]_+ + \frac{\rho_t}{2} \left\| \left[D_k(y - x_k^{(t)}) - q_k^{(t)}\right]_+ \right\|_2^2, \tag{4}$$

  with $\lambda^{(t)} = 0$ in `PC-MALNS`, $\rho_t = 0.35 \cdot 0.985^t$ and the cap $D_k(y - x_k^{(t)}) \le \frac{1}{2}(q_k^{(t)} + s^{(t)})$. Inside the quota the surrogate is the true cost; only the excess is penalized.
- *Level* 3 *(optimistic repair):* the same repair as level 1, from the same random-number state, with cap $s^{(t)}$. This is exactly the single proposal of the optimistic baselines.

Levels 1 to 3 each run $M = 5$ chained destroy–repair steps: the first step repairs the shared x_k^{part}, every later step destroys the level's current solution and repairs it against the remaining cap, and a step is kept only if it lowers the agent's cost. A level that does not lower the cost falls back to $x_k^{(t)}$, so every level satisfies $\Delta c_{k,l}^{(t)} = c_k^\top (y_k^{(t,l)} - x_k^{(t)}) \le 0$. The worker sends only $\Delta c_{k,l}^{(t)}$ and $\Delta D_{k,l}^{(t)} = D_k(y_k^{(t,l)} - x_k^{(t)}) \in \mathbb{R}^m$, $l = 0, \ldots, 3$. The quota-relative usage change is $\delta_{k,l}^{(t)} = \Delta D_{k,l}^{(t)} - q_k^{(t)}$; for a selection $(l_1, \ldots, l_K)$, agent k is a donor on hub r if $\delta_{r,k,l_k}^{(t)} \le 0$ ($k \in \mathbf{K}_r^+$) and a borrower if $\delta_{r,k,l_k}^{(t)} > 0$ ($k \in \mathbf{K}_r^-$). Donor and borrower status is defined per hub: the same agent can lend on one hub and borrow on another.

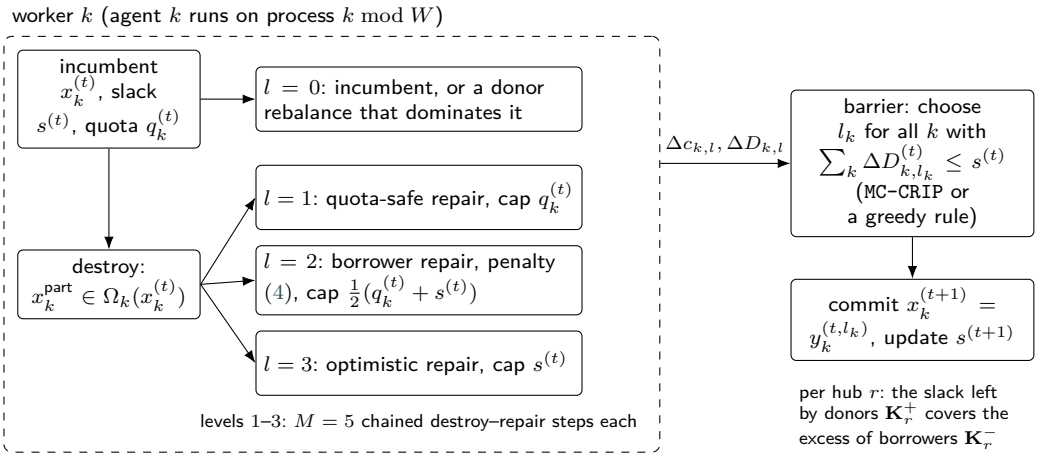


Figure 1: One epoch of PC-MALNS. Level 0 is derived from the incumbent $x_k^{(t)}$; levels 1 to 3 start from the same destroyed solution x_k^{part} and run $M = 5$ chained destroy–repair steps against their capacity caps. Workers send only scalar cost changes and m-dimensional usage changes. The barrier selects one level per worker so that the selection fits into the residual slack; donor and borrower status is defined per hub r.

3.3. Barrier conflict resolution

At the barrier the coordinator first tries the *fast path*: every worker takes its cheapest level $l_k = \arg\min_l \Delta c^{(t)}_{k,l}$, which is committed if $\sum_k \Delta D^{(t)}_{k,l_k} \le s^{(t)}$. Otherwise the epoch is a *conflict epoch* and a commit rule chooses the selection. Because $\sum_k q_k^{(t)} = s^{(t)}$, the quotas cancel in the barrier constraint: for every hub r and selection $(l_1, \dots, l_K)$,

$$\sum_{k=1}^{K} \delta^{(t)}_{r,k,l_k} = \sum_{k=1}^{K} D_{r,k}\big(y_k^{(t,l_k)} - x_k^{(t)}\big) - s_r^{(t)}, \tag{5}$$

so $\sum_k \delta^{(t)}_{r,k,l_k} \le 0$ is equivalent to $\sum_k \Delta D^{(t)}_{r,k,l_k} \le s_r^{(t)}$ whatever the quota shares. Quotas therefore do not restrict the barrier; they act only on the worker side, as trust regions that make levels 1 and 2 different from level 3 and so diversify the portfolio.

Exact barrier (MC-CRIP). With $z_{k,l} \in \{0, 1\}$,

$$\min_z \sum_{k=1}^{K}\sum_{l=0}^{L} \Delta c^{(t)}_{k,l} z_{k,l} \quad \text{s.t.} \quad \sum_{k=1}^{K}\sum_{l=0}^{L} \delta^{(t)}_{r,k,l} z_{k,l} \le 0 \;\; (r = 1, \dots, m), \qquad \sum_{l=0}^{L} z_{k,l} = 1 \;\; (k \in \mathbf{K}). \tag{6}$$

Levels dominated by another level of the same agent are removed. If at most 4,096 level combinations remain, (6) is solved by enumeration, otherwise by HiGHS (Huangfu & Hall, 2018) with relative gap 0 and a 2 s limit. If HiGHS reaches the limit without proving optimality, the coordinator commits the cheapest feasible selection among the HiGHS incumbent, the BCRM-Greedy selection and the all-quota-safe selection $z_{k,1} = 1$. In the experiments this fallback was used in 9 of the 100 XXL and XXXL runs of PC-MALNS [MC-CRIP] in Table 4, once in each of these runs, and never elsewhere.

Greedy barriers. Greedy-Gain-Portfolio and Greedy-Efficiency-Portfolio start every worker at level 0 and scan the upgrades (k, l), $l \ge 1$, with gain $g_{k,l} = \Delta c^{(t)}_{k,0} - \Delta c^{(t)}_{k,l} > 0$ in decreasing order of $g_{k,l}$ or of the efficiency $e_{k,l} = g_{k,l}/(\|[\Delta D^{(t)}_{k,l} - \Delta D^{(t)}_{k,0}]_+\|_1 + 10^{-6})$; an upgrade is committed if the worker is still at level 0 and the extra usage fits into the remaining slack. BCRM-Greedy (quota-surplus greedy upgrade) starts every worker at its quota-safe level 1, which leaves the surplus $S = -\sum_k \delta^{(t)}_{k,1} \ge 0$, and scans the upgrades (k, l), $l \ge 2$, with $\Delta c^{(t)}_{k,l} < \Delta c^{(t)}_{k,1}$ in decreasing order of $\phi_{k,l} = (\Delta c^{(t)}_{k,1} - \Delta c^{(t)}_{k,l})/(\rho_t \|[\beta_{k,l}]_+\|_2^2 + 10^{-6})$ with $\beta_{k,l} = \delta^{(t)}_{k,l} - \delta^{(t)}_{k,1}$; an upgrade is committed if $\beta_{k,l} \le S$, after which $S \leftarrow S - \beta_{k,l}$. All greedy rules take $O(KL \log(KL) + KLm)$ time.

Baseline rules. `Opt-Rollback` commits all cheapest levels if they fit and otherwise one uniformly chosen improving worker. `Opt-SeqLock` checks the cheapest levels in a random arrival order π and commits each while it fits. `Static-Quota` commits level 1 for every worker, and `Demand-Quota` does the same with shares $\alpha_{r,k}$ proportional to agent k's usage of hub r under its unconstrained cheapest patterns. The single-proposal versions of `Opt-Rollback` and `Opt-SeqLock` receive only the optimistic proposal (identical to level 3); the `-Portfolio` versions receive the full portfolio and apply the rule to the cheapest level. Algorithm 1 summarizes one epoch.

Require: feasible incumbent $x^{(t)}$, slack $s^{(t)}$, penalty ρ_t
send $(s^{(t)}, q_k^{(t)} = s^{(t)}/K)$ to the W worker processes (agent k runs on process $k \bmod W$)
for all $k \in \mathbf{K}$ **in parallel do**
  $y_k^{(t,0)} \leftarrow$ donor rebalance of $x_k^{(t)}$ if it dominates $x_k^{(t)}$, else $x_k^{(t)}$
  $x_k^{\text{part}} \leftarrow$ destroy 16%–30% of the requests of $x_k^{(t)}$
  for $l = 1, 2, 3$ **do** ▷ caps $q_k^{(t)}$, $\frac{1}{2}(q_k^{(t)} + s^{(t)})$, $s^{(t)}$
    $y_k^{(t,l)} \leftarrow M$ chained Regret-2 destroy–repair steps from x_k^{part} (level 2 minimizes (4))
    if $c_k^\top y_k^{(t,l)} \geq c_k^\top x_k^{(t)}$ **then** $y_k^{(t,l)} \leftarrow x_k^{(t)}$
  send $(\Delta c_{k,l}^{(t)}, \Delta D_{k,l}^{(t)})_{l=0}^{3}$
$l_k \leftarrow \arg\min_l \Delta c_{k,l}^{(t)}$ for all k
if $\sum_k \Delta D_{k,l_k}^{(t)} \not\leq s^{(t)}$ **then** ▷ conflict epoch
  $(l_1, \ldots, l_K) \leftarrow$ optimal solution of (6) (`MC-CRIP`) or the selection of a greedy rule
  if `MC-CRIP` stopped at its time limit without proof **then** take the cheapest of its incumbent, `BCRM-Greedy` and $l_k = 1\ \forall k$
commit $x_k^{(t+1)} \leftarrow y_k^{(t,l_k)}$ for all k; $s^{(t+1)} \leftarrow d - \sum_k D_k x_k^{(t+1)}$

Algorithm 1: Epoch t of `PC-MALNS`

3.4. Properties

Proposition 1 (Global feasibility, monotone descent and per-epoch commit dominance)**.** *Let $x^{(t)}$ be feasible and let every portfolio contain a level 0 with $\Delta c_{k,0}^{(t)} \leq 0$ and $\Delta D_{k,0}^{(t)} \leq 0$ and a level 1 with $y_k^{(t,1)} \in \mathbf{X}_k$, $\Delta c_{k,1}^{(t)} \leq 0$ and $\Delta D_{k,1}^{(t)} \leq q_k^{(t)}$, where $\sum_k q_k^{(t)} = s^{(t)}$. Then:*

(i) the selection $z_{k,1} = 1$ for all k is feasible for (6);
(ii) every selection that is feasible for (6), in particular the selection of every rule of Section 3.3, gives a feasible $x^{(t+1)}$;
(iii) $C(x^{(t+1)}) \leq C(x^{(t)})$ for every rule, and $C(x^{(t+1)}) \leq C(x^{(t)}) + \sum_k \Delta c_{k,1}^{(t)}$ for `MC-CRIP` and `BCRM-Greedy`;
(iv) for any common portfolio, the cost change of an optimal `MC-CRIP` selection is at most that of `Static-Quota`, `Opt-Rollback`, `Opt-SeqLock` for every arrival order π, `Greedy-Gain-Portfolio`, `Greedy-Efficiency-Portfolio` and `BCRM-Greedy`.

Proof. (i) By (5), $\sum_k \delta_{r,k,1}^{(t)} = \sum_k \Delta D_{r,k,1}^{(t)} - s_r^{(t)} \leq \sum_k q_{r,k}^{(t)} - s_r^{(t)} = 0$. (ii) By (5), feasibility for (6) means $\sum_k D_{r,k} y_k^{(t,l_k)} \leq \sum_k D_{r,k} x_k^{(t)} + s_r^{(t)} = d_r$, and every level lies in $\mathbf{X}_k$. Each rule selects a feasible combination: the fast path and `Opt-Rollback` commit a batch only if it fits, a single improving move fits because its cap is $s^{(t)}$, `Opt-SeqLock` and the greedy rules check the remaining slack before every commit, `Static-Quota` is selection (i), and `BCRM-Greedy` starts from (i) and spends only the surplus S. (iii) Every level has $\Delta c_{k,l}^{(t)} \leq 0$. `MC-CRIP` is optimal and (i) is feasible, so its cost change is at most $\sum_k \Delta c_{k,1}^{(t)}$; `BCRM-Greedy` starts from (i) and accepts only upgrades that lower the cost; the time-limit fallback includes (i). (iv) All rules select a feasible solution of (6) for the same data, and `MC-CRIP` minimizes over all of them. □

Statement (iv) is a per-epoch statement for a common portfolio. Along a run the methods visit different incumbents, so it does not imply that `MC-CRIP` gives the better final solution; Section 5.3 shows that it does not. In our implementation level 3 is bit-for-bit the optimistic proposal of the single-proposal baselines and level 1 the `Static-Quota` proposal, for the same incumbent, seed and epoch (unit tests). The next result

bounds how much a rule can lose against `MC-CRIP` in one epoch. For a rule R and a portfolio, let G_R be the cost reduction of its selection and G^* that of `MC-CRIP`.

Proposition 2 (Worst-case single-epoch ratios)**.** *For every $\varepsilon > 0$ there are portfolios with (a) $G^* > 0$ and $G_{\texttt{Static-Quota}} = 0$; (b) $G^*/\mathbb{E}[G_{\texttt{Opt-Rollback}}] = K - 1$ (expectation over the random single commit); (c) $G^*/G_{\texttt{Opt-SeqLock}}(\pi) = K/(1+\varepsilon)$ for an arrival order π; (d) $G^*/G_{\texttt{Greedy-Gain-Portfolio}} = 2/(1+\varepsilon)$ with one hub and $m/(1+\varepsilon)$ with m hubs; (e) $G^*/G_{\texttt{Greedy-Efficiency-Portfolio}} = C/2$ for every $C > 2$, so the ratio of `Greedy-Efficiency-Portfolio` is unbounded.*

Proof. All portfolios use $q_k = s/K$ and level 0 equal to the incumbent. (a) Two agents, one hub with $s = 1$: agent 1 has a move with gain 10 and usage 1 only at levels 2 and 3, agent 2 has no move; level 1 of agent 1 (cap 1/2) is the incumbent, so `Static-Quota` gains 0 while `MC-CRIP` gains 10. (b) K agents with the same move of gain 1 and usage 1, $s = K - 1$: the batch does not fit, `Opt-Rollback` commits one move, `MC-CRIP` commits $K - 1$. (c) $s = K$; every agent has a level with gain 1 and usage 1, and the first agent in π has a cheapest level with gain $1+\varepsilon$ that uses all of s: `Opt-SeqLock` commits that level and nothing else fits, while `MC-CRIP` commits the K unit moves. (d) One hub with $s = 2$: a move of gain $1+\varepsilon$ and usage 2 is scanned before two moves of gain 1 and usage 1; with m hubs of slack 1, a move of gain $1+\varepsilon$ that uses all hubs blocks m single-hub moves of gain 1. (e) One hub with $s = C$: a move of gain 2 and usage 1 (efficiency 2) is scanned before a move of gain C and usage C (efficiency 1), which then no longer fits. Each construction is verified in `tests/test_worst_case_bounds.py`. □

Proposition 2 gives lower bounds on the worst-case ratios; we do not claim that they are tight. Case (e) is the classical failure of ratio-greedy knapsack heuristics. The bounds concern single epochs; Section 5.3 measures how often and by how much the rules differ on real portfolios.

3.5. Optional prices and quotas (ablation only)

For the ablation of Section 5.5 we keep the price and quota components of the previous version of the method. With *heuristic prices*, $\lambda^{(0)}$ are the LP hub duals and $\lambda^{(t+1)} = [(1-\kappa)\lambda^{(t)} + \gamma_t h^{(t)}]_+$ with $h^{(t)} = \sum_k \Delta D^{(t)}_{k,3} - s^{(t)}$, $\kappa = 0.15$ and $\gamma_t = 0.25 \sum_k |\Delta c^{(t)}_{k,3}|/(\|h^{(t)}\|_2^2 + 1)$. With *`MC-CRIP` LP duals*, $\lambda^{(t+1)} = (1-\kappa')\lambda^{(t)} + \kappa'\mu^{(t)}$, $\kappa' = 0.3$, where $\mu^{(t)}$ are the coupling-row duals of the LP relaxation of (6) in conflict epochs ($\mu^{(t)} = 0$ otherwise). Prices enter the borrower surrogate (4), the donor rebalance and the `BCRM-Greedy` score. *Dynamic quotas* move the shares towards agents whose optimistic move was blocked: with the unrealized gain $g_k = \max\{0, \Delta c^{(t)}_{k,1} - \Delta c^{(t)}_{k,3}\}$ ($g_k = 0$ if level 3 was committed) and the excess $e_{r,k} = [\delta^{(t)}_{r,k,3}]_+$,

$$u_{r,k} = \begin{cases} g_k/e_{r,k} + 10^{-3}, & e_{r,k} > 0, \\ 10^{-3}, & e_{r,k} = 0, \end{cases} \qquad \alpha^{(t+1)}_{r,k} = (1-\omega)\alpha^{(t)}_{r,k} + \omega \frac{u_{r,k}}{\sum_{j=1}^{K} u_{r,j}}, \quad \omega = 0.25. \tag{7}$$

4. Experimental design

Protocol. All configuration choices were made on validation instances (seeds 1 to 10) that are disjoint from the test instances (seeds 101 to 110). A pilot on seeds 1 to 3 compared the starts (`LP-Round`, a decentralized booking start and the direct start with all requests on $p = 0$), the consolidation cap of earlier versions and $M \in \{1, 5\}$; `LP-Round`, no cap and $M = 5$ were best, and on these seeds `MC-CRIP` and `Greedy-Efficiency-Portfolio` tied (mean gaps 2.038% and 2.031%, exact Wilcoxon $p = 0.84$). `MC-CRIP` was therefore fixed as the primary barrier mode before any test run. The parameters were then tuned on seeds 1 to 10 (Section 5.5); no change was adopted.

Instances. RPA-SH instances come from a seeded generator, since no public benchmark exists for this model. The scales are Small, Medium, Large, X-Large, XXL and XXXL with $K = 4, 8, 12, 16, 24, 32$ agents and $N_{\text{agent}} = 30, 40, 50, 60, 100, 125$, i.e. 120 to 4,000 requests. Pattern 0 costs $c_{k,i,0} \sim U(85, 140)$ and uses no capacity. Patterns $p \geq 1$ cost $\theta_{k,i,p} c_{k,i,0}$, where a seeded half of the depots has $\theta \sim U(0.28, 0.52)$ and the other half $\theta \sim U(0.62, 0.85)$; each uses two of $m = 8$ hubs, drawn from the depot's hub profile

Table 3
Compared methods. All LNS methods share workers, operators, random streams and start; they differ in the portfolio and the commit rule.

Method	Candidates per worker	Commit rule
`PC-MALNS [MC-CRIP]` (primary)	levels 0–3	exact (6)
`PC-MALNS [BCRM-Greedy]`	levels 0–3	quota-surplus greedy upgrade
`Greedy-Efficiency-Portfolio`	levels 0–3	greedy by gain per unit of extra usage
`Greedy-Gain-Portfolio`	levels 0–3	greedy by gain
`Opt-Rollback-Portfolio`	levels 0–3	cheapest level, batch rollback
`Opt-SeqLock-Portfolio`	levels 0–3	cheapest level, sequential lock
`Opt-Rollback`	optimistic proposal (level 3)	batch rollback
`Opt-SeqLock`	optimistic proposal (level 3)	sequential lock
`Static-Quota`	quota-safe proposal, $q_k = s^{(t)}/K$	commit all
`Demand-Quota`	quota-safe proposal, demand shares	commit all
`LP-Round`	–	deterministic warm start
HiGHS	–	exact solver, 1 or 4 threads

$\psi_k \sim \mathrm{Dirichlet}(1.5 \cdot \mathbf{1}_m)$, with usage $u(1 + 0.8(1 - \theta))$, $u \sim U(1, 3.5)$, and one of $m_{\mathrm{local}} = 4$ waves with usage $U(1, 2.5)$. With U_r the hub-r usage of the unconstrained cheapest patterns, $d_r = \max\{8, U_r/\eta\}$ for the coupling tightness η, and $b_{k,w} = \max\{4, 0.82 \sum_i A_{w,k,i,p^*_{k,i}}\}$. The main benchmark uses $P = 6$ and $\eta = 1.25$; the rich-pool benchmark (Section 5.2) uses $P \in \{20, 40\}$ on Medium to XXL.

RPA-SH-Sync adds inter-depot transfers. Agents are paired, (k, k') with $k' = k \oplus 1$, and every pair shares a transfer dock with an integer number of slots. For 30% of the requests the last pattern is replaced by a transfer pattern with cost $c_{k,i,0} \cdot U(0.18, 0.32)$ that uses an outbound wave at a hub of agent k, an inbound wave at a different hub of agent k' (usage $1.8 \cdot U(1, 3.5)$ each) and a block of 1, 2 or 3 dock slots (probabilities 0.5, 0.3, 0.2). Every dock has $\max\{1, \lfloor U_T/2 \rfloor\}$ slots, U_T being the slot demand of the cheapest patterns. The design was fixed before any run on these instances.

MMKP benchmarks. The 27 classical instances I07–I13 (Khan et al., 2002) and INST01–INST20 (Hifi et al., 2006) in the distribution of Chen & Hao (2014) have 50 to 400 groups of 10 to 30 items and 10 resources. Each group becomes a request, the groups are split over $K = 8$ agents in contiguous blocks, item j becomes pattern $j + 1$ with cost max profit − profit, and pattern 0 has a cost larger than any profit difference, so that a solution with an unassigned group is worse than every MMKP solution. `LP-Round` leaves at least one group unassigned on 24 of the 27 instances, and no LNS method repairs this, because no worker accepts a cost-increasing move. All LNS methods therefore start from the better of `LP-Round` and the classical two-phase construction of Khan et al. (2002) (lightest item per group, then the best feasible upgrades by profit per added normalized weight); its time is charged. On INST04 neither start is feasible. Gaps are measured against the best known values of verified solution certificates.

Methods and budgets. Table 3 lists the methods. All LNS methods share the worker code, the operators ($M = 5$, destroy 16% to 30%, Regret-2 with noise $\sigma = 2.0$ and 2.7 in a best-of-two repair), the random stream of worker k in epoch t and run $\iota \in \{0, \ldots, 4\}$, and the `LP-Round` start. They run on a persistent pool of $W = 4$ worker processes; process w owns the agents k with $k \bmod W = w$, keeps their incumbents and portfolios, and exchanges only the per-level cost and usage changes with the coordinator, so the trajectories are identical to serial execution. The wall-clock budgets are 0.5, 1, 1.5, 2, 4 and 6 s from Small to XXXL and include the `LP-Round` time and shipping the instance to the processes; the one-time process start-up (below 0.5 s) is not charged. HiGHS 1.8.0 runs in a separate process with relative gap 10^{-4}. The reference run uses one thread and 600 s (Small, Medium) or 60 s (Large to XXXL) in the main benchmark, 30 s in the rich-pool benchmark and the tightness sweep, and 60 s on RPA-SH-Sync; HiGHS (5s), HiGHS (60s) and HiGHS (CPU-matched) are read from the incumbent trace of this run. HiGHS (wall-matched) runs separately with one and with four threads and a time limit equal to the mean wall-clock time of the five `PC-MALNS [MC-CRIP]` runs of the instance; HiGHS (CPU-matched) uses their mean CPU time. HiGHS times cover the solver call; building and passing the model took at most 0.04 s.

Measurements. The gap is $100(C(x) - LB)/LB$ with $LB = \max\{\text{LP bound, dual bound of the reference HiGHS run}\}$; no reference run proved optimality, not even at 600 s on Small, so the gaps are upper bounds on the true gaps. The primal integral (Berthold, 2013) over the run's wall-clock time uses the primal gap $|C - LB|/\max\{|C|, |LB|\}$, set to 1 before the first solution. `PC-MALNS` has no solution until the `LP-Round` start exists and has been shipped to the processes, which takes on average 0.01 to 0.23 s with $P = 6$, up to 0.36 s with $P = 20$ and up to 0.67 s with $P = 40$ (XXL). We record wall-clock time, CPU time (start plus the sum of worker and barrier times), conflict epochs, and the acceptance rate (committed improving moves per worker with an improving candidate).

Statistics. The unit of analysis is the instance: the mean of the five runs of an LNS method, and the single value of a deterministic method. For every comparison we report the paired mean difference with Student-t and bootstrap (10,000 resamples) 95% confidence intervals, wins, ties and losses, and the exact two-sided Wilcoxon signed-rank test (zero differences dropped, mid-ranks, exact conditional null distribution, never a normal approximation), Holm-adjusted within each table. Experiments ran one job at a time on an Apple M4 (4 performance and 6 efficiency cores, 16 GB) with Python 3.11, NumPy 2.3.5, SciPy 1.16.3 and highspy 1.8.0.

5. Results

5.1. Main benchmark

Table 4 reports the mean gaps per scale and Table 5 the pooled comparison with `PC-MALNS` [`MC-CRIP`]. Starting from `LP-Round` (2.90% over all 60 instances), `PC-MALNS` [`MC-CRIP`] reaches 1.51%. It beats `Opt-Rollback` and `Opt-SeqLock` on 58 and 56 of the 60 instances (mean differences −0.169 and −0.145 percentage points, pp) and their portfolio versions on 57 instances each (−0.172 and −0.131 pp), all with $p_{\text{Holm}} < 10^{-4}$. Giving the optimistic rules the four-level portfolio therefore does not close the gap: the loss comes from the commit rule, not from the candidates. `Static-Quota` and `Demand-Quota` lose on all 60 instances (−0.929 and −0.730 pp). Among the portfolio barriers, `PC-MALNS` [`MC-CRIP`] is slightly better than `BCRM-Greedy` (40/0/20, −0.012 pp, $p_{\text{Holm}} = 0.0336$) and `Greedy-Gain-Portfolio` (46/0/14), but `Greedy-Efficiency-Portfolio` is better than `PC-MALNS` [`MC-CRIP`] on 44 of 60 instances (+0.026 pp, 95% CI $[0.010, 0.043]$, $p_{\text{Holm}} < 10^{-4}$); Section 5.3 analyses this result.

Against HiGHS the picture depends on the budget (Figure 2). With the same wall-clock time, `PC-MALNS` [`MC-CRIP`] is better than HiGHS with one thread on 44 of 60 instances (−0.570 pp, $p_{\text{Holm}} = 0.0089$) and with four threads on 41 (−0.443 pp, $p_{\text{Holm}} = 0.0336$). The advantage is confined to Large and above: on Small and Medium, HiGHS is better already at 0.5 to 1 s. HiGHS with 5 s is better on Small to X-Large, but on XXL and XXXL its incumbent at 5 s (1.74% and 1.46%) is still worse than that of `PC-MALNS`; HiGHS with 60 s and HiGHS with the same CPU time are better on all 60 instances. HiGHS with four threads hardly differs from one thread, as its branch and bound is essentially sequential. The primal integral of wall-matched HiGHS is lower than that of `PC-MALNS` on the larger scales despite its worse final gap, because HiGHS finds a first, weak incumbent almost immediately while `PC-MALNS` has no solution before its start exists (Figure 3).

5.2. Rich candidate pools

With 20 or 40 candidates per request (Table 6, Figure 4), the integer program grows to 20 or 40 binaries per request, and HiGHS with the same wall-clock time does not improve its incumbent within these limits (its CPU-matched incumbent is the same). `PC-MALNS` [`MC-CRIP`] reaches 2.26% ($P = 20$) and 2.25% ($P = 40$), against 3.81% and 4.11% for wall-matched HiGHS, and is better on 40 of 40 instances for each P, with one or four threads and also against HiGHS with the same CPU time ($p_{\text{Holm}} < 10^{-4}$). HiGHS with 30 s is better at $P = 20$ (11/0/29, $p_{\text{Holm}} = 0.0026$) and not significantly different at $P = 40$ (2.72% against 2.25%, $p_{\text{Holm}} = 0.4680$). As on $P = 6$, `Greedy-Efficiency-Portfolio` is slightly better than `PC-MALNS` [`MC-CRIP`] (+0.051 and +0.033 pp) and `PC-MALNS` [`MC-CRIP`] beats the optimistic rules on 38 to 40 of 40 instances. The primal-integral comparison favours wall-matched HiGHS (Table 6): `PC-MALNS` pays 0.04 to 0.67 s of start-up during which its primal gap is 1 by definition, which outweighs its better final gap within budgets of 1 to 4 s. We therefore claim an advantage in final gap, not in primal integral.

Table 4
Main benchmark ($P = 6$, $\eta = 1.25$, 10 test instances per scale): mean gap to the lower bound (%) per scale and mean primal integral (PI, s × gap fraction) over all 60 instances. LNS methods: mean over 5 runs per instance on $W = 4$ processes with wall-clock budgets of 0.5, 1, 1.5, 2, 4 and 6 s. HiGHS (5s), (60s) and (CPU-matched) are read from the incumbent trace of the 1-thread reference run (600 s on Small and Medium, 60 s otherwise); the wall-matched runs are separate runs with the mean wall-clock time of PC-MALNS [MC-CRIP].

Method	Small	Medium	Large	X-Large	XXL	XXXL	All	PI
LP-Round	5.11	3.67	3.05	2.61	1.62	1.31	2.90	–
PC-MALNS [MC-CRIP]	2.55	1.83	1.66	1.41	0.90	0.73	1.51	0.108
PC-MALNS [BCRM-Greedy]	2.57	1.87	1.65	1.42	0.91	0.74	1.53	0.106
Greedy-Efficiency-Portfolio	2.57	1.80	1.58	1.38	0.88	0.72	1.49	0.105
Greedy-Gain-Portfolio	2.62	1.83	1.66	1.45	0.95	0.80	1.55	0.107
Opt-Rollback-Portfolio	2.75	2.04	1.82	1.57	1.05	0.88	1.69	0.112
Opt-SeqLock-Portfolio	2.63	2.03	1.81	1.57	1.00	0.82	1.64	0.109
Opt-Rollback	2.74	2.04	1.80	1.58	1.05	0.88	1.68	0.110
Opt-SeqLock	2.62	2.05	1.81	1.60	1.02	0.84	1.66	0.108
Static-Quota	4.18	3.11	2.61	2.23	1.39	1.14	2.44	0.120
Demand-Quota	3.82	2.87	2.41	2.04	1.29	1.04	2.24	0.117
HiGHS (1T, wall-matched)	1.31	1.56	3.40	3.05	1.74	1.46	2.08	0.054
HiGHS (4T, wall-matched)	1.31	1.42	3.03	2.78	1.74	1.46	1.96	0.054
HiGHS (1T, CPU-matched)	1.11	0.69	0.47	0.40	0.23	0.18	0.51	0.109
HiGHS (5s)	0.90	0.69	0.47	0.45	1.74	1.46	0.95	0.081
HiGHS (60s)	0.59	0.42	0.36	0.34	0.22	0.16	0.35	0.318
HiGHS (600s)	0.47	0.32	–	–	–	–	0.40	2.686

Table 5
Main benchmark, pooled over the 60 test instances: PC-MALNS [MC-CRIP] (mean gap 1.514%) against every method. Diff = PC-MALNS [MC-CRIP] − method in percentage points (negative favours PC-MALNS [MC-CRIP]) with paired-t and bootstrap 95% confidence intervals; W/T/L counts the instances on which PC-MALNS [MC-CRIP] is better, equal or worse; p: exact two-sided Wilcoxon signed-rank test, Holm-adjusted over the 16 comparisons.

Method	Gap	Diff	t 95% CI	Bootstrap 95% CI	W/T/L	p_{Holm}
LP-Round	2.896	−1.382	[−1.578, −1.187]	[−1.574, −1.200]	60/0/0	<0.0001
PC-MALNS [BCRM-Greedy]	1.526	−0.012	[−0.026, +0.001]	[−0.025, +0.001]	40/0/20	0.0336
Greedy-Efficiency-Portfolio	1.487	+0.026	[+0.010, +0.043]	[+0.009, +0.041]	16/0/44	<0.0001
Greedy-Gain-Portfolio	1.550	−0.036	[−0.057, −0.015]	[−0.057, −0.016]	46/0/14	0.0012
Opt-Rollback-Portfolio	1.686	−0.172	[−0.200, −0.144]	[−0.201, −0.145]	57/0/3	<0.0001
Opt-SeqLock-Portfolio	1.645	−0.131	[−0.156, −0.107]	[−0.154, −0.107]	57/0/3	<0.0001
Opt-Rollback	1.683	−0.169	[−0.200, −0.139]	[−0.199, −0.140]	58/0/2	<0.0001
Opt-SeqLock	1.659	−0.145	[−0.172, −0.118]	[−0.172, −0.119]	56/0/4	<0.0001
Static-Quota	2.443	−0.929	[−1.050, −0.808]	[−1.046, −0.814]	60/0/0	<0.0001
Demand-Quota	2.244	−0.730	[−0.826, −0.634]	[−0.823, −0.639]	60/0/0	<0.0001
HiGHS (1T, wall-matched)	2.084	−0.570	[−0.887, −0.254]	[−0.871, −0.261]	44/0/16	0.0089
HiGHS (4T, wall-matched)	1.956	−0.443	[−0.760, −0.125]	[−0.746, −0.133]	41/0/19	0.0336
HiGHS (1T, CPU-matched)	0.512	+1.001	[+0.907, +1.096]	[+0.912, +1.096]	0/0/60	<0.0001
HiGHS (5s)	0.951	+0.562	[+0.304, +0.820]	[+0.309, +0.810]	20/0/40	<0.0001
HiGHS (60s)	0.348	+1.166	[+1.036, +1.295]	[+1.043, +1.293]	0/0/60	<0.0001
HiGHS (600s)	0.398	+1.792	[+1.617, +1.966]	[+1.638, +1.954]	0/0/20	<0.0001

5.3. Commit-rule isolation: exact versus greedy barriers

Table 7 compares the commit rules on Medium and Large under the `LP-Round` start, under the direct start with all requests on $p = 0$, which raises the share of conflict epochs to 26.5%, and on RPA-SH-Sync. Three results hold in all three regimes. First, `PC-MALNS [MC-CRIP]` beats every optimistic rule, with or without the portfolio, on 19 or 20 of 20 instances (under the direct start by 1.3 to 2.9 pp). Second, `PC-MALNS [MC-CRIP]` has the lowest conflict rate of the portfolio rules. Third, `Greedy-Efficiency-Portfolio` finishes better than `PC-MALNS [MC-CRIP]`: by 0.052 pp under the `LP-Round` start ($p_{\text{Holm}} = 0.0283$), 0.443 pp under the direct start

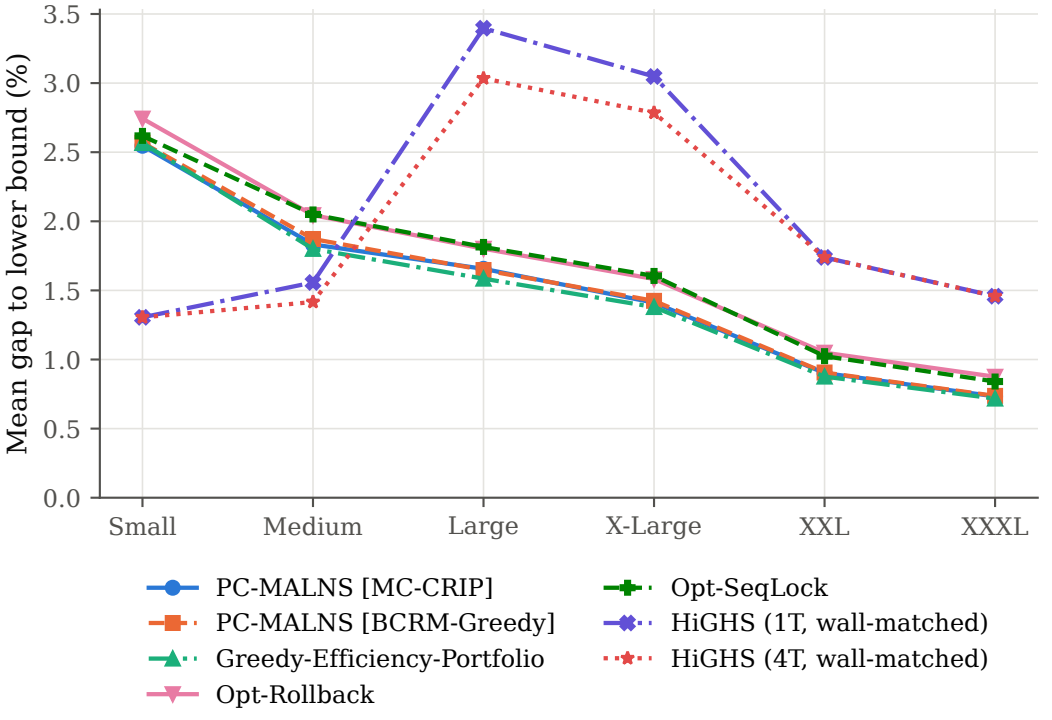


Figure 2: Mean gap to the lower bound by scale ($P = 6$, $\eta = 1.25$; LNS methods: mean over 5 runs per instance on $W = 4$ processes; HiGHS with the same wall-clock time).

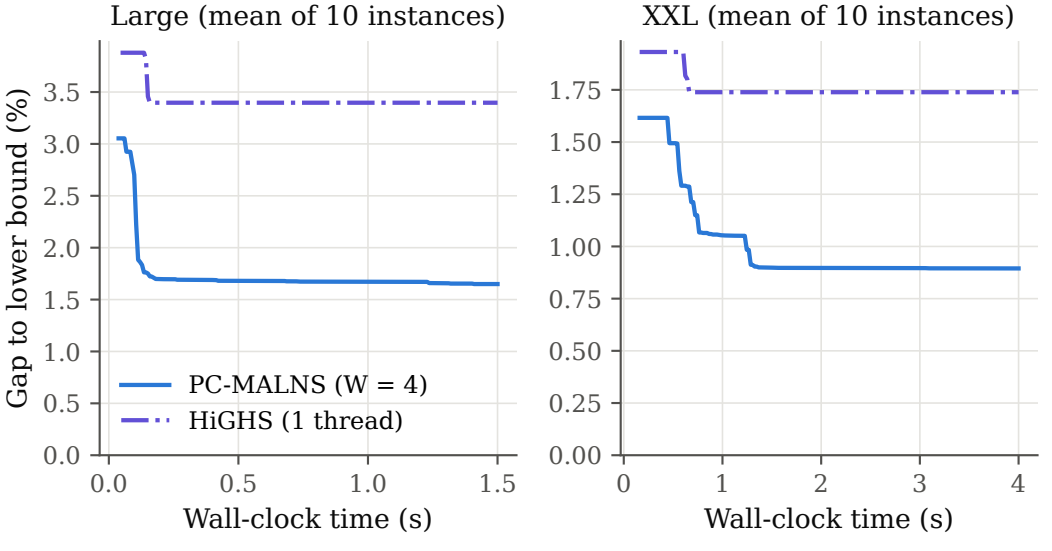


Figure 3: Anytime behaviour on Large and XXL ($P = 6$): mean gap of `PC-MALNS [MC-CRIP]` (run 0, $W = 4$) and of the 1-thread HiGHS reference run over wall-clock time. Each curve starts once every instance has a solution.

($p_{\text{Holm}} = 0.0002$) and 0.084 pp on RPA-SH-Sync ($p_{\text{Holm}} = 0.0043$). Under the direct start `BCRM-Greedy` is also better than `MC-CRIP`, while `MC-CRIP` beats `Greedy-Gain-Portfolio` (15/0/5). The transfer-synchronised

Table 6
Rich candidate pools ($P \in \{20, 40\}$, 10 test instances per scale and P): mean gap (%) per scale and over all 40 instances (All), mean primal integral (PI), and the pooled comparison of PC-MALNS [MC-CRIP] with each method (Diff in percentage points, W/T/L, exact Wilcoxon p Holm-adjusted within each P). Wall-clock budgets as in Table 4; HiGHS (30s) is the 1-thread reference run.

Method	Medium	Large	X-Large	XXL	All	PI	Diff	W/T/L	p_{Holm}
$P = 20$									
LP-Round	4.50	3.66	3.01	1.89	3.26	–	−1.001	40/0/0	<0.0001
PC-MALNS [MC-CRIP]	3.00	2.51	2.14	1.40	2.26	0.195			
PC-MALNS [BCRM-Greedy]	3.03	2.52	2.15	1.37	2.27	0.195	−0.004	21/0/19	0.6656
Greedy-Efficiency-Portfolio	2.91	2.49	2.09	1.36	2.21	0.194	+0.051	5/0/35	<0.0001
Opt-Rollback	3.21	2.79	2.39	1.55	2.49	0.198	−0.223	39/0/1	<0.0001
Opt-SeqLock	3.21	2.83	2.38	1.56	2.50	0.197	−0.232	39/0/1	<0.0001
HiGHS (1T, wall-matched)	5.38	4.27	3.44	2.16	3.81	0.063	−1.549	40/0/0	<0.0001
HiGHS (4T, wall-matched)	5.38	4.27	3.44	2.16	3.81	0.063	−1.549	40/0/0	<0.0001
HiGHS (1T, CPU-matched)	5.38	4.27	3.44	2.16	3.81	0.229	−1.549	40/0/0	<0.0001
HiGHS (30s)	1.79	2.07	0.67	0.95	1.37	0.727	+0.895	11/0/29	0.0026
$P = 40$									
LP-Round	4.62	3.51	2.90	1.77	3.20	–	−0.957	40/0/0	<0.0001
PC-MALNS [MC-CRIP]	3.03	2.47	2.11	1.37	2.25	0.323			
PC-MALNS [BCRM-Greedy]	3.02	2.47	2.18	1.37	2.26	0.322	−0.014	26/0/14	0.1450
Greedy-Efficiency-Portfolio	2.95	2.42	2.12	1.36	2.21	0.322	+0.033	14/0/26	0.0167
Opt-Rollback	3.23	2.73	2.41	1.55	2.48	0.325	−0.234	38/0/2	<0.0001
Opt-SeqLock	3.28	2.79	2.38	1.55	2.50	0.325	−0.251	40/0/0	<0.0001
HiGHS (1T, wall-matched)	5.72	4.44	3.85	2.41	4.11	0.062	−1.861	40/0/0	<0.0001
HiGHS (4T, wall-matched)	5.72	4.44	3.85	2.44	4.11	0.062	−1.869	40/0/0	<0.0001
HiGHS (1T, CPU-matched)	5.72	4.44	3.85	2.41	4.11	0.236	−1.861	40/0/0	<0.0001
HiGHS (30s)	3.69	3.56	1.23	2.41	2.72	0.968	−0.477	28/0/12	0.4680

Table 7
Commit-rule isolation (Medium and Large, 20 test instances per regime, 5 runs per instance). All rules see the same workers and operators; the portfolio rules see the same four-level portfolio. Gap: mean gap (%); Conf.: share of epochs in which the cheapest levels do not fit (%); Diff: PC-MALNS [MC-CRIP] − rule (percentage points; negative favours MC-CRIP); W/T/L of PC-MALNS [MC-CRIP]; *: exact Wilcoxon $p < 0.05$ after Holm adjustment within the regime.

	LP-Round start				Direct start				RPA-SH-Sync (LP-Round start)			
Rule	Gap	Conf.	Diff	W/T/L	Gap	Conf.	Diff	W/T/L	Gap	Conf.	Diff	W/T/L
PC-MALNS [MC-CRIP]	1.74	7.7			5.98	26.5			2.12	13.4		
PC-MALNS [BCRM-Greedy]	1.76	10.6	−0.016	15/0/5	5.73	32.4	+0.257	3/0/17*	2.12	17.7	−0.002	9/0/11
Greedy-Efficiency-Portfolio	1.69	10.3	+0.052	7/0/13*	5.54	31.0	+0.443	4/0/16*	2.03	16.8	+0.084	5/0/15*
Greedy-Gain-Portfolio	1.75	9.3	−0.001	11/0/9	6.21	30.8	−0.222	15/0/5*	2.13	15.5	−0.009	12/0/8
Opt-Rollback-Portfolio	1.93	27.9	−0.186	20/0/0*	8.92	80.1	−2.934	20/0/0*	2.33	46.3	−0.209	20/0/0*
Opt-SeqLock-Portfolio	1.92	12.9	−0.178	20/0/0*	7.77	34.5	−1.786	20/0/0*	2.28	19.0	−0.164	19/0/1*
Opt-Rollback	1.92	11.4	−0.179	20/0/0*	7.27	36.5	−1.285	19/0/1*	2.32	18.5	−0.198	20/0/0*
Opt-SeqLock	1.93	5.6	−0.188	20/0/0*	7.56	15.8	−1.578	20/0/0*	2.30	8.1	−0.183	20/0/0*

instances, designed to challenge greedy rules with paired, lumpy dock capacities, do not change the ranking: `MC-CRIP` ties `BCRM-Greedy` and `Greedy-Gain-Portfolio` and loses to `Greedy-Efficiency-Portfolio`.

Table 8 explains why this is surprising. On each of the 1,955 conflict epochs of the `MC-CRIP` runs, every rule was applied to the same portfolios. `MC-CRIP` is strictly better than `Greedy-Efficiency-Portfolio` in about half of these epochs and than `Greedy-Gain-Portfolio` in 38% to 43%; `Greedy-Efficiency-Portfolio` obtains only 83% to 84% of the `MC-CRIP` cost reduction, so the exact barrier gains about 20% more per conflict epoch. The optimistic rules obtain 11% to 86% and `Static-Quota` as little as 6% (Figure 5). The per-epoch dominance of Proposition 1(iv) is thus large in practice, yet it does not carry over to the final solutions.

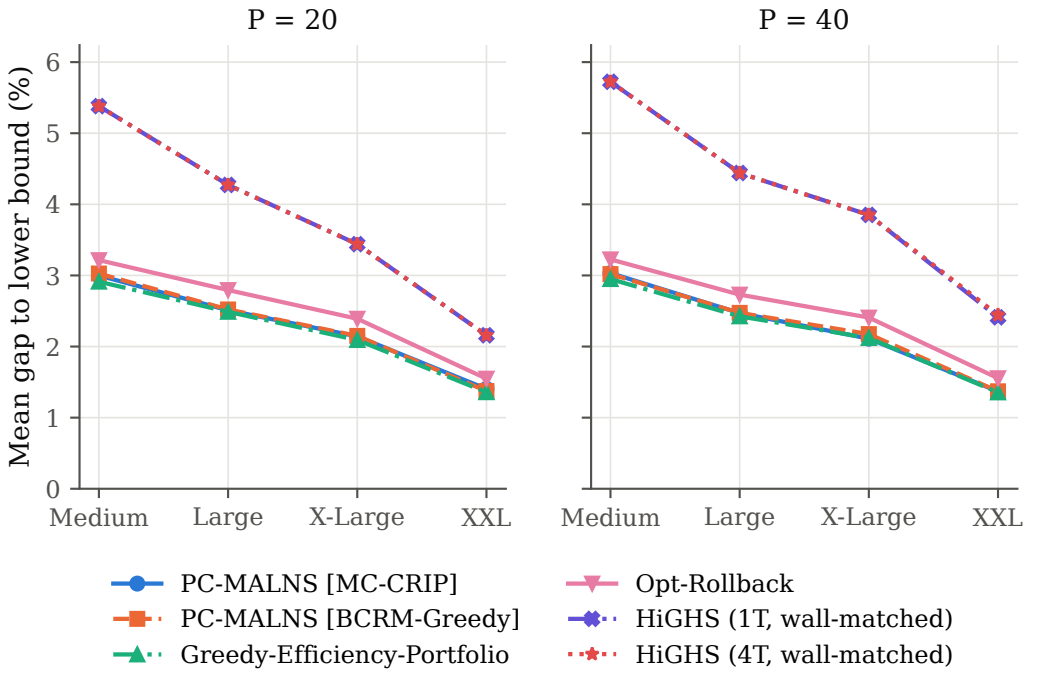


Figure 4: Mean gap with rich candidate pools ($P = 20$ and $P = 40$). HiGHS runs with the mean wall-clock time of `PC-MALNS [MC-CRIP]`; one and four threads give almost identical results.

Table 8
Counterfactual barrier analysis. On every conflict epoch of the PC-MALNS [MC-CRIP] runs of Table 7 (409, 1,109 and 437 epochs, 1,955 in total), every rule is applied to the same portfolios. Share: total cost reduction of the rule as a percentage of the MC-CRIP reduction; Strict: percentage of these epochs in which MC-CRIP is strictly better. Counterfactual evaluations use their own random stream and are not charged to any clock.

	LP-Round start		Direct start		RPA-SH-Sync	
Rule	Share	Strict	Share	Strict	Share	Strict
BCRM-Greedy	79.6	58.2	95.5	54.3	76.9	61.3
Greedy-Efficiency	84.0	51.3	84.1	47.6	83.2	52.9
Greedy-Gain	83.9	43.0	93.1	37.8	84.3	39.4
Opt-SeqLock	57.4	100.0	85.9	100.0	60.4	99.8
Opt-Rollback	20.6	100.0	10.6	100.0	19.4	100.0
Static-Quota	6.3	100.0	74.5	100.0	5.8	100.0

Mechanism. To see why per-epoch optimality does not pay off, we reran `PC-MALNS [MC-CRIP]`, `BCRM-Greedy` and `Greedy-Efficiency-Portfolio` on the instances of Table 7 with the same budgets and logged the residual hub slack after every epoch (Table 9; these are separate runs, and 80% of the `LP-Round` runs reproduce Table 4 exactly, the others differ by 0.004 pp on average because wall-clock budgets shift the last epoch). `MC-CRIP` keeps less slack than `Greedy-Efficiency-Portfolio` on 19 of 20 instances under both starts (time-averaged mean slack 0.54% against 0.61% of capacity with the warm start, 2.51% against 3.08% with the direct start; exact Wilcoxon $p < 10^{-4}$), less slack on the scarcest hub on 14 and 16 of 20 instances ($p = 0.005$ and 0.004), and it commits about a quarter fewer improving moves per epoch (0.33 against 0.41 and 1.10 against 1.36). This is consistent with the following explanation: in a conflict epoch `MC-CRIP` packs the residual slack into the combination of moves with the largest immediate gain, which often includes capacity-hungry moves, whereas sorting by gain per unit of capacity spends slack on frugal moves first and leaves capacity that later epochs, with new destroyed neighborhoods, turn into further gains. The

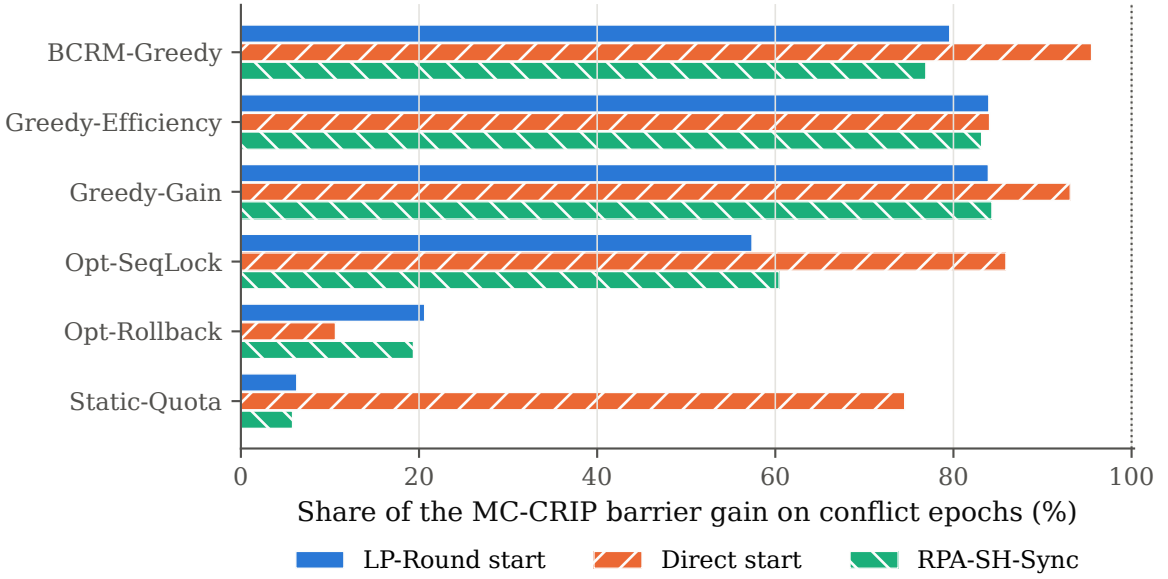


Figure 5: Share of the `MC-CRIP` cost reduction obtained by each rule on the same portfolios, over all conflict epochs of the `PC-MALNS [MC-CRIP]` runs in Table 7.

Table 9
Residual hub slack along the trajectory (Medium and Large, 20 test instances, 5 runs each; separate diagnostic runs with the budgets of Table 7). Slack: time average over the epochs of $\frac{1}{m}\sum_r s_r^{(t)}/d_r$ (%); Bottleneck: time average of $\min_r s_r^{(t)}/d_r$ (%); Acc./epoch: committed improving worker moves per epoch; Conf.: conflict epochs (%).

Rule	Gap	Slack	Bottleneck	Epochs	Acc./epoch	Conf.
LP-Round start						
PC-MALNS [MC-CRIP]	1.745	0.54	0.068	58.2	0.33	7.2
PC-MALNS [BCRM-Greedy]	1.760	0.61	0.082	59.1	0.43	9.6
Greedy-Efficiency-Portfolio	1.693	0.61	0.080	58.8	0.41	9.3
Direct start						
PC-MALNS [MC-CRIP]	5.923	2.51	2.062	51.7	1.10	23.8
PC-MALNS [BCRM-Greedy]	5.688	2.67	2.034	53.3	1.34	27.9
Greedy-Efficiency-Portfolio	5.480	3.08	2.249	52.9	1.36	26.7

diagnostic shows an association, not a causal proof; a barrier that values residual slack, for instance by a look-ahead penalty, would be needed to test it directly.

We keep `PC-MALNS [MC-CRIP]` as the primary mode in all tables because it was fixed on the validation instances, on which it tied `Greedy-Efficiency-Portfolio`; choosing `Greedy-Efficiency-Portfolio` after seeing the test results would be a selection on test data. The practical recommendation is nevertheless clear: the cheap efficiency rule is at least as good as the exact barrier at no solver cost, and a new validation study should confirm it as the default.

5.4. Public MMKP benchmarks: a structural limit

Table 10 reports the MMKP results. All LNS methods end between 2.59% and 2.91% from the best known solutions, against 0.20% for HiGHS with the same wall-clock time and 0.13% with 60 s. `PC-MALNS` improves its start on 17 of the 26 instances with a feasible start (mean gap from 2.91% to 2.59%) and then stalls; conflict epochs occur in fewer than 0.5% of the epochs, so the barrier rules hardly differ, and only `Static-Quota` is significantly worse. The reason is structural. Every worker filters out candidates that raise its own cost ($\Delta c_{k,l}^{(t)} > 0$), which guarantees monotone descent (Proposition 1(iii)); once the start has nearly filled the knapsack capacities, an improvement needs some groups to switch to lighter, more expensive items

Table 10
Public MMKP benchmarks (I07–I13 and INST01–INST20, groups split over $K = 8$ agents; 2 s wall clock on $W = 4$ processes; 5 runs per instance). Gap to the best known solution (%, MMKP-feasible runs only), share of MMKP-feasible runs (%), conflict epochs (%), and the comparison of PC-MALNS [MC-CRIP] with each method (26 instances with a feasible start; Holm-adjusted exact Wilcoxon).

Method	I07–I13	INST01–20	All	Feasible	Conf.	Diff	W/T/L	p_{Holm}
PC-MALNS [MC-CRIP]	2.948	2.463	2.594	96.3	0.38			
PC-MALNS [BCRM-Greedy]	2.980	2.481	2.615	96.3	0.44	−0.022	7/15/4	0.5244
Opt-Rollback	2.919	2.513	2.622	96.3	0.29	−0.029	8/11/7	0.6056
Opt-SeqLock	2.961	2.466	2.599	96.3	0.21	−0.006	7/12/7	0.7148
Static-Quota	3.131	2.828	2.909	96.3	0.00	−0.316	17/9/0	<0.0001
HiGHS (1T, wall-matched)	0.089	0.233	0.195	100.0	–	+2.406	0/0/26	<0.0001
HiGHS (4T, wall-matched)	0.089	0.233	0.195	100.0	–	+2.407	0/0/26	<0.0001
HiGHS (60s)	0.049	0.152	0.125	100.0	–	+2.475	0/0/26	<0.0001

Table 11
Coupling tightness sweep (Large, 10 test instances per η; $\eta = 1.25$ from Table 4): mean gap (%).

Method	$\eta = 0.85$	$\eta = 1.05$	$\eta = 1.25$	$\eta = 1.45$
LP-Round	2.93	3.06	3.05	1.71
PC-MALNS [MC-CRIP]	1.80	1.84	1.66	1.11
PC-MALNS [BCRM-Greedy]	1.80	1.85	1.65	1.13
Greedy-Efficiency-Portfolio	1.79	1.85	1.58	1.10
Opt-Rollback	1.70	1.82	1.80	1.21
Opt-SeqLock	1.70	1.79	1.81	1.22
Static-Quota	1.74	2.55	2.61	1.63
HiGHS (1T, wall-matched)	0.95	3.62	3.40	0.45
HiGHS (1T, CPU-matched)	0.32	0.47	0.47	0.41

so that others can upgrade. No worker proposes such a sacrifice, with or without prices, so the search cannot trade capacity between agents. Dedicated MMKP methods (Chen & Hao, 2014; Gao et al., 2017; Mansini & Zanotti, 2020; Lamanna et al., 2022) reach the best known values; we report these instances as a boundary of the method, not as a competitive benchmark.

5.5. Tightness, ablation, tuning and scaling

Coupling tightness. Table 11 and Figure 6 vary η on Large. With loose coupling ($\eta = 0.85$), the optimistic single-proposal rules are better than `PC-MALNS` (1.70% against 1.80%, 9 and 8 of 10 instances, unadjusted $p = 0.006$ and 0.014); the portfolio pays off once the hubs bind ($\eta \geq 1.25$), where `PC-MALNS` beats them on 9 or 10 of 10 instances. At $\eta \neq 1.25$, `Greedy-Efficiency-Portfolio` and `PC-MALNS` [`MC-CRIP`] do not differ significantly. With 40 tests and 10 instances per η, the smallest attainable Holm-adjusted p-value is 0.076, so no comparison of the sweep is significant after adjustment; this family was fixed before the runs.

Prices and quotas. Table 12 adds the price and quota components of Section 3.5 to `PC-MALNS` [`MC-CRIP`]. None of the six variants differs significantly from the default ($p_{\text{Holm}} = 1.0$ for all); five are nominally better by 0.02 to 0.03 pp (on 6 of 10 instances) and the dynamic quota without prices is nominally worse. Because the barrier constraint does not depend on the quotas (5) and the portfolio already offers the barrier a quota-safe, a borrowing and an optimistic version of every move, steering the borrower repair by prices or reallocating quotas adds no information that the barrier does not already use. We therefore treat prices and dynamic quotas as optional components and omit them from `PC-MALNS`.

Parameters. Table 13 reports the one-factor-at-a-time sensitivity on the validation seeds. No level qualified to replace the default under the pre-declared rule; $M = 10$ came closest ($p_{\text{Holm}} = 0.083$), and a single step ($M = 1$) was worst. Results are insensitive to the destroy range, the noise and ρ_0 within the tested ranges.

Parallel scaling. Table 14 and Figure 7 report the measured wall clock for a fixed number of epochs. Four processes give a speedup of 2.18 to 2.37 over serial execution and eight processes 2.76 to 2.86, with

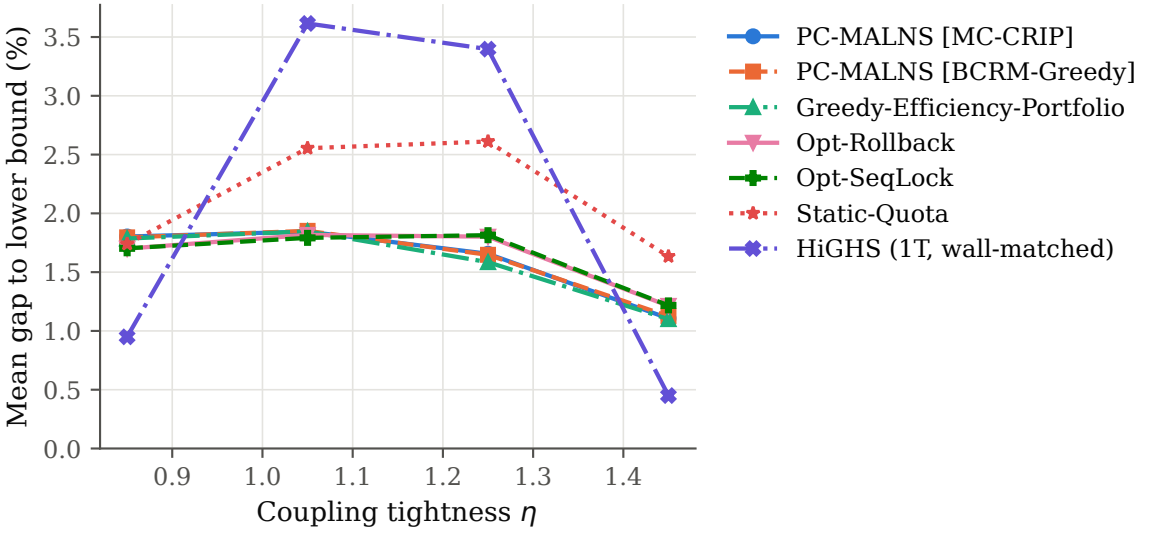


Figure 6: Mean gap on Large as a function of the coupling tightness η.

Table 12
Price and quota ablation of PC-MALNS [MC-CRIP] (Large, $\eta = 1.25$, 10 test instances, 5 runs each): mean gap (%), difference default − variant, W/T/L of the default, exact Wilcoxon p Holm-adjusted over the six variants.

Variant	Gap	Diff	W/T/L	p_{Holm}
No prices, static quota (default)	1.656			
Heuristic prices, dynamic quota	1.629	+0.028	4/0/6	1.0000
Heuristic prices, static quota	1.632	+0.024	4/0/6	1.0000
MC-CRIP LP duals, dynamic quota	1.627	+0.029	4/0/6	1.0000
MC-CRIP LP duals, static quota	1.635	+0.021	4/0/6	1.0000
No prices, demand quota	1.634	+0.022	4/0/6	1.0000
No prices, dynamic quota	1.662	−0.006	7/1/2	1.0000

Table 13
Parameter sensitivity on the validation seeds 1–10 (Medium, 3 runs each, 2 s CPU budget, one factor at a time around the default in bold): level: mean gap (%) (Holm-adjusted exact Wilcoxon p against the default over 30 paired runs). A level would replace the default only if it were better with $p_{\text{Holm}} < 0.05$; no level qualified.

Factor	Levels
Chained steps M	1: 1.938 (0.0832); 2: 1.910 (0.1579); **5: 1.851**; 10: 1.799 (0.0832)
Destroy fraction range	[0.08, 0.16]: 1.818 (0.4733); **[0.16, 0.3]: 1.851**; [0.3, 0.45]: 1.908 (0.4733)
Regret-2 noise σ	0.0: 1.846 (1.0000); 1.0: 1.863 (1.0000); **2.0: 1.851**; 4.0: 1.793 (0.2197)
Borrower penalty ρ_0	0.0: 1.874 (1.0000); 0.15: 1.872 (1.0000); **0.35: 1.851**; 0.7: 1.813 (1.0000)

identical final solutions. The simulated parallel clock (maximum worker time per epoch) would allow 7 to 8; the difference comes from inter-process communication, shipping the instance, load imbalance between processes and the sequential barrier. The efficiency cores of the machine also run the processes beyond the fourth. Figure 8 shows the acceptance rates: the portfolio barriers commit 50% to 62% of the improving proposals, the optimistic rules 19% to 41%, and the quota rules all of them, but only small ones.

5.6. Discussion

The study supports three practical conclusions. First, when concurrent LNS workers share capacities, letting each worker offer a few capacity-stratified alternatives and resolving them jointly is clearly better than optimistic commit or quotas; the size of the effect (0.13 to 0.17 pp with the warm start, 1.3 to 2.9 pp from

Table 14
Measured wall clock (s) of PC-MALNS [MC-CRIP] for a fixed number of epochs (Large: 40, XXL: 20; seeds 101–103), serial in one process and on W worker processes (speedup against serial). Every run gives the same final objective as the serial run. Jobs were run one at a time on an Apple M4 (4 performance and 6 efficiency cores).

Scale	Serial	$W = 1$	$W = 2$	$W = 4$	$W = 8$
Large	3.12	3.16 (0.99×)	1.83 (1.72×)	1.32 (2.37×)	1.13 (2.76×)
XXL	5.64	5.59 (1.01×)	3.62 (1.56×)	2.59 (2.18×)	1.97 (2.86×)

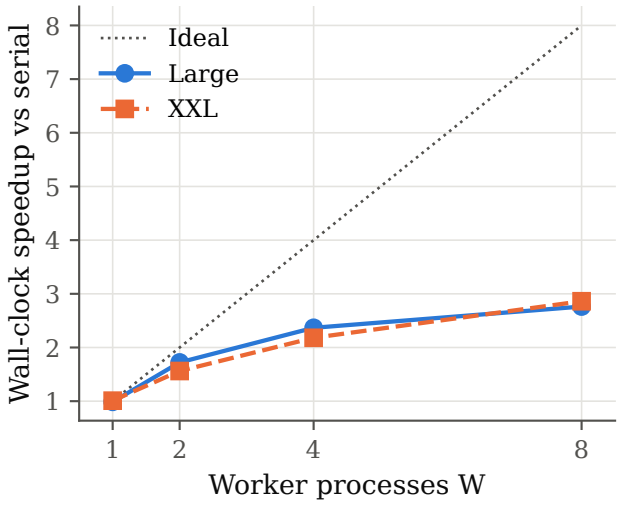


Figure 7: Measured wall-clock speedup of PC-MALNS [MC-CRIP] against serial execution for a fixed number of epochs (Table 14).

the cold start) grows with the share of conflict epochs. Second, how the barrier is resolved matters much less than offering the alternatives: an efficiency-sorted greedy pass is as good as or better than the exact program over whole runs, although it is worse in individual epochs. Third, against an exact solver the approach is competitive only under hard real-time limits and when the integer program is large relative to the budget, as with 20 or 40 candidates per request; with a few seconds more, HiGHS is better on compact instances. The main limitation is the improvement-only worker: it guarantees monotone descent but cannot release capacity at a local loss, which the MMKP instances expose. A coordinator that asks for such sacrifices, for example through capacity-buying levels priced by the barrier, is the most promising extension.

6. Conclusion

We studied the synchronization barrier of concurrent multi-agent LNS for block-angular multiple-choice allocation with shared capacities. In PC-MALNS, workers send four capacity-stratified candidates and the barrier selects one per worker. Every epoch is feasible with monotone descent; the quotas cancel in the barrier constraint and serve only to diversify the portfolio; and the exact barrier dominates every other rule in each epoch, while greedy rules can be arbitrarily worse in a single epoch. In a controlled study with real multi-process execution, five runs per instance and exact tests, portfolio coordination beat optimistic rollback and sequential-lock commit on 56 to 58 of 60 instances, also when these rules received the same portfolio, and beat static and demand-proportional quotas on all 60. Counterfactual logging showed that the exact barrier gains about 20% more per conflict epoch than greedy efficiency resolution, yet the greedy rule finished slightly better over whole runs in every family at the default tightness. Under budgets of 0.5 to 6 s on four processes, the method beat wall-clock-matched HiGHS on all 80 instances with 20 or 40 candidates per request and on the larger six-candidate instances, whereas HiGHS was better with 5 s or

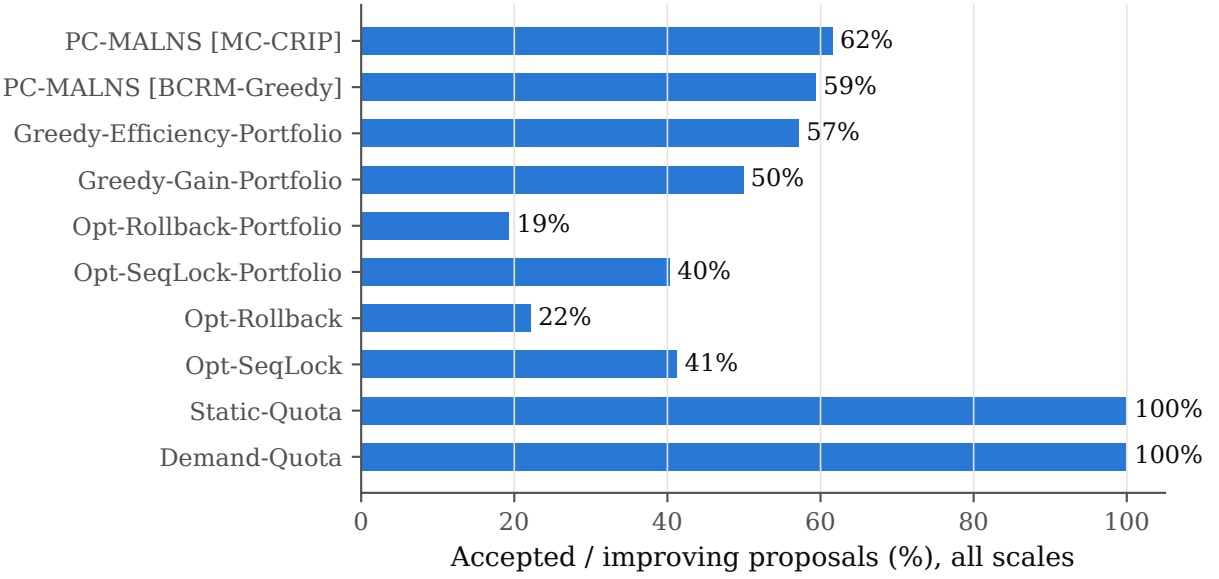


Figure 8: Committed improving moves per worker with an improving candidate (Table 4, all scales). The axis ends at 105%.

more on most compact instances and with equal CPU time on all of them. Prices and dynamic quotas gave no measurable gain, and the public MMKP instances revealed that improvement-only workers cannot trade capacity across agents. Future work should study barrier rules that look beyond the current epoch, workers that offer capacity-releasing sacrifices, and coarser tasks that reduce communication overhead.

CRediT authorship contribution statement

Naresh R: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic) and Gemini (Google) for language polishing, LaTeX formatting, and code debugging. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Declaration of competing interest

The author declares that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The source code, the RPA-SH and RPA-SH-Sync instance generators, the MMKP conversion, all raw result files, the unit tests and the scripts that generate every table, figure and number of this paper are publicly available at `https://github.com/Naresh23032003/DG-MALNS`.

A. Implementation details

All details are implemented in the public repository, in the modules `operators`, `worker`, `commit_rules`, `bcrm_solver`, `engine` and `agent_pool` of the `src` directory.

- *Destroy.* Worker k removes $n_{\mathrm{d}} = \max\{2, \mathrm{round}(f N_{\mathrm{agent}})\}$ requests with $f \sim U(0.16, 0.30)$. First, $\min\{|C_k|, \max\{1, \lfloor 0.45\, n_{\mathrm{d}} \rfloor\}\}$ consolidated requests ($C_k = \{i : p_i \geq 1\}$) with the largest score $2\sum_w A_{w,k,i,p_i} / \max\{1, c_{k,i,0} - c_{k,i,p_i}\} + U(0, 0.25)$ are removed; the remaining removals are the requests with the largest score $(c_{k,i,p_i} - \min_p c_{k,i,p}) + U(0.5, 18) + U(0, 22)$. Removed requests are set to $p = 0$.
- *Regret-2 repair.* Each destroyed request gets the perturbed cost $\tilde{c}_{i,p} = c_{k,i,p} - c_{k,i,0} + \varepsilon_{i,p}$, $\varepsilon_{i,p} \sim N(0, \sigma^2)$, $\tilde{c}_{i,0} = 0$, with $\sigma = 2.0$ and $\sigma = 2.7$ for the two repairs of a best-of-two repair. In each step, for every unassigned request the candidates that respect the level's cap and b_k are ranked; with best value v_1 and second-best value v_2, the request with the largest score $0.7(-v_1) + 0.3(v_2 - v_1)$ receives its best candidate. The borrower level adds the increase of the penalty in (4) to the value of a candidate. There is no limit on the number of consolidations per step.
- *Chained steps.* Step $j \geq 2$ of a level destroys the level's current solution with the same destroy operator and repairs it against the cap minus the usage already consumed by the level; it is kept only if it lowers the agent's cost.
- *Donor rebalance.* With hub weights $\nu = \mathbf{1}$ (or $\nu = \lambda^{(t)}$ with prices), up to two consolidated requests with the largest weighted burden per unit of saving, $\nu^\top D_{k,i,p_i} / (c_{k,i,0} - c_{k,i,p_i} + 1)$, move to their cheapest pattern p' with $\nu^\top D_{k,i,p'} < \nu^\top D_{k,i,p_i}$; then one other request receives the upgrade with the largest saving whose added weighted usage does not exceed the released usage, subject to b_k.
- *Barrier.* `MC-CRIP` removes dominated levels and fixes agents with one remaining level; with at most 4,096 combinations it enumerates, otherwise it calls HiGHS with relative gap 0 and a 2 s limit, with the fallback of Section 3.3. Ties in the greedy rules are broken by agent and level index.
- *Counterfactual logging.* In each conflict epoch of a `PC-MALNS` [`MC-CRIP`] run, all rules are applied to the portfolios of the epoch; `Opt-Rollback` is evaluated by its exact expectation and `Opt-SeqLock` by the mean over 20 random arrival orders, with a separate random stream. The logging time is subtracted from all clocks.
- *Tolerances.* All feasibility checks use the absolute tolerance 10^{-6}, and every committed selection and every final solution is verified.

References

Alba, E. (2005). *Parallel Metaheuristics: A New Class of Algorithms.* John Wiley & Sons.

Asokan, B. V., Furman, K. C., Goel, V., Li, G., & Shao, Y. (2014). Parallel large-neighborhood search techniques for LNG inventory routing. *Optimization Online*, 4319.

Bartodziej, P., Derigs, U., & Vogel, U. (2010). On the potentials of parallelizing large neighbourhood search for rich vehicle routing problems. In *Learning and Intelligent Optimization (LION 4)*, Lecture Notes in Computer Science 6073 (pp. 216–219). Springer.

Berthold, T. (2013). Measuring the impact of primal heuristics. *Operations Research Letters*, 41(6), 611–614.

Bertsekas, D. P. (1982). *Constrained Optimization and Lagrange Multiplier Methods.* Academic Press, New York.

Blum, C., Pinacho, P., López-Ibáñez, M., & Lozano, J. A. (2016). Construct, merge, solve & adapt: A new general algorithm for combinatorial optimization. *Computers & Operations Research*, 68, 75–88.

Boyd, S., Parikh, N., Chu, E., Peleato, B., & Eckstein, J. (2011). Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1), 1–122.

Chan, S.-H., Chen, Z., Lin, D.-L., Zhang, Y., Harabor, D., Koenig, S., Huang, T.-W., & Phan, T. (2024). Anytime multi-agent path finding using operation parallelism in large neighborhood search. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2024)* (pp. 2183–2185).

Chen, Y., & Hao, J.-K. (2014). A "reduce and solve" approach for the multiple-choice multidimensional knapsack problem. *European Journal of Operational Research*, 239(2), 313–322.

Crainic, T. G. (2019). Parallel metaheuristics and cooperative search. In *Handbook of Metaheuristics* (pp. 419–451). Springer.

Falsone, A., Margellos, K., Garatti, S., & Prandini, M. (2017). Dual decomposition for multi-agent distributed optimization with coupling constraints. *Automatica*, 84, 149–158.

Fioretto, F., Dovier, A., Pontelli, E., Yeoh, W., & Zivan, R. (2017). Solving DCOPs with distributed large neighborhood search. *arXiv preprint arXiv:1702.06915*.

Fioretto, F., Pontelli, E., & Yeoh, W. (2018). Distributed constraint optimization problems and applications: A survey. *Journal of Artificial Intelligence Research*, 61, 623–698.

Fisher, M. L. (1981). The Lagrangian relaxation method for solving integer programming problems. *Management Science*, 27(1), 1–18.

Gao, C., Lu, G., Yao, X., & Li, J. (2017). An iterative pseudo-gap enumeration approach for the multidimensional multiple-choice knapsack problem. *European Journal of Operational Research*, 260(1), 1–11.

Geoffrion, A. M. (1970). Primal resource-directive approaches for optimizing nonlinear decomposable systems. *Operations Research*, 18(3), 375–403.

Groër, C., Golden, B., & Wasil, E. (2011). A parallel algorithm for the vehicle routing problem. *INFORMS Journal on Computing*, 23(2), 315–330.

Guignard, M. (2003). Lagrangean relaxation. *TOP*, 11(2), 151–200.

Held, M., Wolfe, P., & Crowder, H. P. (1974). Validation of subgradient optimization. *Mathematical Programming*, 6(1), 62–88.

Hemmelmayr, V. C., Cordeau, J.-F., & Crainic, T. G. (2012). An adaptive large neighborhood search heuristic for two-echelon vehicle routing problems arising in city logistics. *Computers & Operations Research*, 39(12), 3215–3228.

Hendel, G. (2022). Adaptive large neighborhood search for mixed integer programming. *Mathematical Programming Computation*, 14(2), 185–221.

Hifi, M., Michrafy, M., & Sbihi, A. (2004). Heuristic algorithms for the multiple-choice multidimensional knapsack problem. *Journal of the Operational Research Society*, 55(12), 1323–1332.

Hifi, M., Michrafy, M., & Sbihi, A. (2006). A reactive local search-based algorithm for the multiple-choice multi-dimensional knapsack problem. *Computational Optimization and Applications*, 33(2–3), 271–285.

Hoang, K. D., Fioretto, F., Yeoh, W., Pontelli, E., & Zivan, R. (2018). A large neighboring search schema for multi-agent optimization. In *Principles and Practice of Constraint Programming (CP 2018)* (pp. 688–706). Springer.

Huangfu, Q., & Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119–142.

Joncour, C., Michel, S., Sadykov, R., Sverdlov, D., & Vanderbeck, F. (2010). Column generation based primal heuristics. *Electronic Notes in Discrete Mathematics*, 36, 695–702.

Khan, S., Li, K. F., Manning, E. G., & Akbar, M. M. (2002). Solving the knapsack problem for adaptive multimedia systems. *Studia Informatica Universalis*, 2(1), 157–178.

Lahrichi, N., Crainic, T. G., Gendreau, M., Rei, W., Crisan, G. C., & Vidal, T. (2015). An integrative cooperative search framework for multi-decision-attribute combinatorial optimization: Application to the MDPVRP. *European Journal of Operational Research*, 246(2), 400–412.

Lamanna, L., Mansini, R., & Zanotti, R. (2022). A two-phase kernel search variant for the multidimensional multiple-choice knapsack problem. *European Journal of Operational Research*, 297(1), 53–65.

Li, J., Chen, Z., Harabor, D., Stuckey, P. J., & Koenig, S. (2021). Anytime multi-agent path finding via large neighborhood search. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI 2021)* (pp. 4127–4135).

Mansini, R., & Zanotti, R. (2020). A core-based exact algorithm for the multidimensional multiple choice knapsack problem. *INFORMS Journal on Computing*, 32(4), 1061–1079.

Masson, R., Lehuédé, F., & Péton, O. (2013). An adaptive large neighborhood search for the pickup and delivery problem with transfers. *Transportation Science*, 47(3), 344–355.

Pisinger, D., & Røpke, S. (2007). A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34(8), 2403–2435.

Polyak, B. T. (1969). Minimization of unsmooth functionals. *USSR Computational Mathematics and Mathematical Physics*, 9(3), 14–29.

Potvin, J.-Y., & Rousseau, J.-M. (1993). A parallel route building algorithm for the vehicle routing and scheduling problem with time windows. *European Journal of Operational Research*, 66(3), 331–340.

Røpke, S. (2009). PALNS: A software framework for parallel large neighborhood search. In *Proceedings of the 8th Metaheuristics International Conference (MIC 2009)*, CD-ROM, pp. id-1–id-10.

Røpke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472.

Sadykov, R., Vanderbeck, F., Pessoa, A., Tahiri, I., & Uchoa, E. (2019). Primal heuristics for branch and price: The assets of diving methods. *INFORMS Journal on Computing*, 31(2), 251–267.

Santini, A., Røpke, S., & Hvattum, L. M. (2018). A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic. *Journal of Heuristics*, 24(5), 783–815.

Schryen, G. (2020). Parallel computational optimization in operations research: A new integrative framework, literature review and research directions. *European Journal of Operational Research*, 287(1), 1–18.

Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming (CP 1998)* (pp. 417–431). Springer.

Vujanic, R., Mohajerin Esfahani, P., Goulart, P. J., Mariéthoz, S., & Morari, M. (2016). A decomposition method for large scale MILPs, with performance guarantees and a power system application. *Automatica*, 67, 144–156.